Live Coding Python

Mini-Exemples — Systèmes Distribués

Séance 5 — Applications Réparties et Cybersécurité

Module : Applications Réparties
Pré-requis : Python, notions réseau, JSON

25 avril 2026

Table des matières

1 Introduction **3**

2 Exemple 1a — Serveur HTTP Minimal (`serveur.py`) **3**
2.1 Objectif pédagogique . . . 3
2.2 Concepts abordés . . . 3
2.3 Code source . . . 3
2.4 Points pédagogiques . . . 5
2.5 Résultat d'exécution . . . 5

3 Exemple 1b — Client HTTP avec Timeout (`client.py`) **6**
3.1 Objectif pédagogique . . . 6
3.2 Concepts abordés . . . 6
3.3 Code source . . . 6
3.4 Points pédagogiques . . . 7
3.5 Résultat d'exécution . . . 7

4 Exemple 2 — Retry avec Backoff Exponentiel (`retry.py`) **8**
4.1 Objectif pédagogique . . . 8
4.2 Concepts abordés . . . 8
4.3 Formule du délai . . . 8
4.4 Code source . . . 8
4.5 Points pédagogiques . . . 10
4.6 Résultat d'exécution . . . 10

5 Exemple 3 — Appels Asynchrones avec `asyncio` (`async_demo.py`) **11**
5.1 Objectif pédagogique . . . 11
5.2 Concepts abordés . . . 11
5.3 Impact sur la latence perçue . . . 11
5.4 Code source . . . 11
5.5 Points pédagogiques . . . 12
5.6 Résultat d'exécution . . . 13

6 Synthèse Comparative **14**
6.1 Pièges fréquents à éviter . . . 14

7 Questions de Révision **14**

1 Introduction

Cette section présente quatre extraits de code Python exécutables illustrant les concepts fondamentaux des systèmes distribués : communication client-serveur, gestion du timeout, résilience par retry, et optimisation de la latence par appels asynchrones.

Chaque exemple est accompagné d'une analyse pédagogique et d'un espace réservé à l'insertion de captures d'écran des résultats d'exécution.

N°	Fichier	Concept clé	Bibliothèque
1a	`serveur.py`	Communication client-serveur	`http.server`
1b	`client.py`	Timeout, gestion d'erreurs	`urllib`, `socket`
2	`retry.py`	Résilience, backoff, jitter	`urllib`, `random`
3	`async_demo.py`	Parallélisme, latence	`asyncio`

Table 1 – Vue d'ensemble des quatre exemples

2 Exemple 1a — Serveur HTTP Minimal (`serveur.py`)

2.1 Objectif pédagogique

Implémenter un serveur HTTP en Python pur (sans framework) servant des ressources JSON. Cet exemple illustre le **modèle client-serveur** et la **sérialisation JSON**, deux briques fondamentales de toute communication dans un système distribué.

2.2 Concepts abordés

- **Communication client-serveur** : échange de requêtes et réponses entre deux processus distincts.
- **Latence variable** : un délai aléatoire (entre 0 et 2 secondes) simule les fluctuations réelles du réseau.
- **Sérialisation JSON** : conversion d'un objet Python en flux d'octets transmissible sur le réseau.
- **Codes de retour HTTP** : `200 OK`, `404 Not Found`, `405 Method Not Allowed`.

2.3 Code source

```python
#!/usr/bin/env python3
"""
Mini serveur HTTP qui renvoie des documents JSON.
Utilise uniquement la stdlib Python (http.server + json).
"""

import json
import time
import random
from http.server import HTTPServer, BaseHTTPRequestHandler

```

```python
# Base de "documents" en memoire (simule une DB)
DOCUMENTS = {
    "1": {"id": 1, "titre": "Rapport Q3",       "auteur": "Alice"},
    "2": {"id": 2, "titre": "Plan strategique", "auteur": "Bob"},
    "3": {"id": 3, "titre": "Audit securite",   "auteur": "Charlie"},
}


class DocumentHandler(BaseHTTPRequestHandler):

    def log_message(self, format, *args):
        """Affichage epure des requetes recues."""
        print(f"[SERVEUR] {self.address_string()} - {format % args}")

    def do_GET(self):
        # Simuler une latence reseau variable (0 a 2 secondes)
        latence = random.uniform(0, 2.0)
        print(f"Latence simulee : {latence:.2f}s")
        time.sleep(latence)

        # Routage : /documents  ou  /documents/{id}
        path  = self.path.strip("/")
        parts = path.split("/")

        if parts[0] == "documents":
            if len(parts) == 1:
                self._send_json(200, list(DOCUMENTS.values()))
            elif len(parts) == 2 and parts[1] in DOCUMENTS:
                self._send_json(200, DOCUMENTS[parts[1]])
            else:
                self._send_json(404, {"error": "Document non trouve"})
        else:
            self._send_json(404, {"error": "Route inconnue"})

    def do_POST(self):
        """Methode non supportee dans cette version."""
        self._send_json(405, {"error": "Methode non supportee"})

    def _send_json(self, status_code, data):
        """Envoie une reponse JSON avec le code HTTP donne."""
        response = json.dumps(data, ensure_ascii=False).encode("utf-8")
        self.send_response(status_code)
        self.send_header("Content-Type",
                         "application/json; charset=utf-8")
        self.send_header("Content-Length", str(len(response)))
        self.end_headers()
        self.wfile.write(response)


if __name__ == "__main__":
    serveur = HTTPServer(("127.0.0.1", 8000), DocumentHandler)
```

```python
    print("Serveur demarre sur http://127.0.0.1:8000")
    print("  Essayez : /documents  ou  /documents/1")
    serveur.serve_forever()
```

Listing 1 – serveur.py — Mini serveur HTTP de documents JSON

2.4 Points pédagogiques

— Le `time.sleep(random.uniform(0, 2.0))` simule la **latence variable** du réseau : parfois quasi-instantanée, parfois jusqu'à 2 secondes.

— Le dictionnaire `DOCUMENTS` agit comme une base de données en mémoire — point de départ avant d'introduire une vraie persistance.

— Aucune authentification n'est implémentée : c'est volontaire pour isoler la notion de communication avant celle de sécurité.

2.5 Résultat d'exécution

```
PS C:\Users\imada> cd Downloads
PS C:\Users\imada\Downloads> python Server.py
Serveur demarre sur http://127.0.0.1:8000
  Essayez : /documents  ou  /documents/1
Latence simulee : 0.36s
[SERVEUR] 127.0.0.1 - "GET /documents/2 HTTP/1.1" 200 -
```

FIGURE 1 – Exécution de `serveur.py` : latence variable simulée visible pour chaque requête entrante.

3 Exemple 1b — Client HTTP avec Timeout (`client.py`)

3.1 Objectif pédagogique

Montrer comment un client HTTP doit **impérativement** définir un timeout sur chaque appel réseau. Sans timeout, un appel vers un service lent bloque le thread appelant indéfiniment, épuisant progressivement les ressources du système.

3.2 Concepts abordés

— **Timeout** : délai maximal d'attente avant abandon de la requête.

— **Gestion des exceptions réseau** : distinguer timeout, connexion refusée et autres erreurs.

— **Désérialisation JSON** : reconstruction de l'objet Python à partir de la réponse.

— **Variabilité observée** : sur plusieurs appels, certains réussissent, d'autres expirent — c'est la réalité du réseau.

3.3 Code source

```python
#!/usr/bin/env python3
"""
Mini client HTTP qui appelle le serveur de documents.
Utilise uniquement la stdlib Python (urllib + socket).
Montre la gestion du timeout et des erreurs reseau.
"""

import json
import time
import socket
import urllib.request
import urllib.error

BASE_URL        = "http://127.0.0.1:8000"
TIMEOUT_SECONDS = 1.5   # On abandonne si le serveur met plus de 1.5 s


def get_document(doc_id):
    """Recupere un document par son ID avec gestion du timeout."""
    url = f"{BASE_URL}/documents/{doc_id}"
    print(f"\nRequete vers {url} (timeout={TIMEOUT_SECONDS}s)")

    debut = time.time()
    try:
        response = urllib.request.urlopen(url, timeout=TIMEOUT_SECONDS)
        duree    = time.time() - debut

        raw  = response.read().decode("utf-8", errors="replace")
        data = json.loads(raw)
        print(f"Reponse recue en {duree:.2f}s")
        print(f"  Donnees : {data}")
```

```python
        return data

    except (urllib.error.URLError, socket.timeout) as e:
        duree = time.time() - debut
        msg   = str(e)
        if "timed out" in msg or isinstance(e, socket.timeout):
            print(f"TIMEOUT apres {duree:.2f}s - serveur trop lent !")
        else:
            print(f"Erreur reseau apres {duree:.2f}s : {e}")
        return None


if __name__ == "__main__":
    # Cinq appels consecutifs pour observer la variabilite
    for i in range(5):
        get_document("1")
```

Listing 2 – client.py — Client HTTP avec gestion du timeout

3.4 Points pédagogiques

— Sur 5 appels, certains réussissent (latence inférieure à 1,5 s) et d'autres expirent (latence supérieure à 1,5 s). C'est exactement la réalité des systèmes distribués : **la latence est imprévisible**.

— `socket.timeout` est capturé en plus de `urllib.error.URLError` car selon la version de Python et du système d'exploitation, l'exception peut remonter directement.

— `errors="replace"` dans `decode()` rend le décodage robuste face à des octets invalides.

A retenir

Toujours configurer un **timeout** sur les appels réseau. Un appel sans timeout peut bloquer indéfiniment un thread et provoquer un *resource starvation*, suivi d'une cascade de pannes dans le reste du système.

3.5 Résultat d'exécution

```
PS C:\Users\imada> cd Downloads
PS C:\Users\imada\Downloads> python client.py

Requete vers http://127.0.0.1:8000/documents/1 (timeout=1.5s)
TIMEOUT apres 1.57s - serveur trop lent !

Requete vers http://127.0.0.1:8000/documents/1 (timeout=1.5s)
TIMEOUT apres 1.51s - serveur trop lent !
```

FIGURE 2 – Exécution de `client.py` : variabilité de la latence sur 5 appels — certains réussissent, d'autres expirent en timeout.

4 Exemple 2 — Retry avec Backoff Exponentiel (`retry.py`)

4.1 Objectif pédagogique

Implémenter le pattern **retry + backoff exponentiel + jitter** : réessayer automatiquement un appel réseau défaillant en espaçant progressivement les tentatives afin de ne pas surcharger un service fragilisé.

4.2 Concepts abordés

— **Retry** : réessayer automatiquement en cas d'erreur transitoire (réseau instable, service en redémarrage).

— **Backoff exponentiel** : le délai d'attente double à chaque échec, réduisant la pression sur le service.

— **Jitter** : variation aléatoire ajoutée au délai pour éviter que tous les clients retentent simultanément (*thundering herd problem*).

4.3 Formule du délai

Pour chaque tentative : Le délai est calculé selon la formule : le délai de base multiplié par 2 à la puissance de l'index de la tentative, auquel on ajoute une valeur aléatoire (le jitter) pour éviter les collisions.

Tentative	Index de tentative	Délai de base	Délai max (avec jitter)
1 (immédiate)	—	0 s	0 s
2	0	0,5 s	0,65 s
3	1	1,0 s	1,30 s
4 (dernière)	2	2,0 s	2,60 s

Table 2 – Progression des délais avec `base_delay = 0.5` et `max_retries = 4`

4.4 Code source

```python
#!/usr/bin/env python3
"""
Implementation d'un retry avec backoff exponentiel + jitter.
Pattern essentiel en systemes distribues pour la resilience.
"""

import json
import time
import random
import urllib.request
import urllib.error


def fetch_with_retry(url, max_retries=4, base_delay=0.5, timeout=1.5):
```

```python
    """
    Appelle ‘url‘ avec retry + backoff exponentiel + jitter.

    Strategie de backoff (base_delay = 0.5, max_retries = 4) :
      - Tentative 1 : immediate
      - Tentative 2 : ~0.50 s  (base_delay * 2^0 + jitter)
      - Tentative 3 : ~1.00 s  (base_delay * 2^1 + jitter)
      - Tentative 4 : ~2.00 s  (base_delay * 2^2 + jitter)

    Le jitter evite le "thundering herd" : si tous les clients
    retentent en meme temps, le serveur re-crash immediatement.

    Returns:
        dict  si succes, None si toutes les tentatives echouent.
    """
    for tentative in range(max_retries):
        try:
            print(f"  Tentative {tentative + 1}/{max_retries}...")
            response = urllib.request.urlopen(url, timeout=timeout)
            data     = json.loads(
                response.read().decode("utf-8", errors="replace")
            )
            print(f"  Succes a la tentative {tentative + 1}")
            return data

        except (urllib.error.HTTPError, urllib.error.URLError) as e:
            print(f"  Echec : {e}")

            if tentative < max_retries - 1:
                delay  = base_delay * (2 ** tentative)
                jitter = random.uniform(0, delay * 0.3)
                wait   = delay + jitter
                print(f"  Attente {wait:.2f}s avant retry...")
                time.sleep(wait)

    print("  Toutes les tentatives ont echoue.")
    return None


if __name__ == "__main__":
    print("Appel avec retry + backoff exponentiel :")
    result = fetch_with_retry("http://127.0.0.1:8000/documents/2")
    if result:
        print(f"\nDocument recupere : {result}")
    else:
        print("\nImpossible de joindre le service.")
```

Listing 3 – retry.py — Retry avec backoff exponentiel et jitter

4.5 Points pédagogiques

— Sans jitter, 1 000 clients qui échouent en même temps et retentent tous après exactement 1 s provoquent un re-crash immédiat du serveur.

— `HTTPError` est capturée avant `URLError` car elle en est une sous-classe : cela prépare le code à traiter les deux cas séparément si nécessaire.

— Le `return None` après la boucle signale explicitement l'échec total, facilitant la lecture et les tests unitaires.

A retenir

Ne jamais réessayer immédiatement en boucle (*tight retry*) : c'est l'équivalent d'un DDoS involontaire sur le service malade. Le backoff exponentiel avec jitter est le standard de l'industrie.

4.6 Résultat d'exécution

```
PS C:\Users\imada\Downloads> python Time.py
Appel avec retry + backoff exponentiel :
  Tentative 1/4...
  Succes a la tentative 1

Document recupere : {'id': 2, 'titre': 'Plan strategique', 'auteur': 'Bob'}
PS C:\Users\imada\Downloads>
```

FIGURE 3 – Exécution de `retry.py` : tentatives successives avec backoff exponentiel et jitter.

5 Exemple 3 — Appels Asynchrones avec `asyncio` (`async_demo.py`)

5.1 Objectif pédagogique

Comparer quantitativement les appels **séquentiels** et les appels **parallèles** : en distribué, appeler plusieurs services l'un après l'autre additionne les latences, alors qu'avec `asyncio.gather` le temps total est réduit au maximum des latences individuelles.

5.2 Concepts abordés

- **Coroutine (`async def`)** : fonction suspendable par la boucle d'événements pendant les E/S.
- **`await`** : cède le contrôle à la boucle le temps de l'attente réseau.
- **`asyncio.gather`** : exécute plusieurs coroutines en parallèle et attend leur terminaison collective.

5.3 Impact sur la latence perçue

Le temps séquentiel est la somme des latences individuelles, tandis que le temps parallèle correspond à la latence maximale parmi les appels.

Avec Auth = 0,5 s, Stockage = 0,8 s, Recherche = 0,3 s :

Temps séquentiel = 0,5 + 0,8 + 0,3 = 1,6 s, contre Temps parallèle = max(0,5, 0,8, 0,3) = 0,8 s (soit un gain de 50 %).

5.4 Code source

```python
#!/usr/bin/env python3
"""
Illustration : appels sequentiels vs asynchrones.
Montre l'impact direct de la latence en systeme distribue :
  - Sequentiel : les latences s'additionnent
  - Parallele  : le temps total = max des latences individuelles
"""

import asyncio
import time


async def appeler_service(nom, latence):
    """Simule un appel a un service distant avec une latence donnee."""
    print(f"  Appel a {nom} (latence prevue : {latence}s)...")
    await asyncio.sleep(latence)   # Simule l'attente reseau
    print(f"  {nom} a repondu !")
    return {"service": nom, "status": "ok"}


async def appels_sequentiels():
    """Appels l'un apres l'autre --- les latences s'additionnent."""
```

```python
    print("\n-- Appels SEQUENTIELS --")
    debut = time.time()

    r1 = await appeler_service("Auth",      0.5)
    r2 = await appeler_service("Stockage",  0.8)
    r3 = await appeler_service("Recherche", 0.3)

    total = time.time() - debut
    print(f"  Temps total sequentiel : {total:.2f}s")
    print(f"  Attendu : 0.5 + 0.8 + 0.3 = 1.6s")
    return [r1, r2, r3]


async def appels_paralleles():
    """Appels en parallele --- temps total = max des latences."""
    print("\n-- Appels PARALLELES (asyncio.gather) --")
    debut = time.time()

    resultats = await asyncio.gather(
        appeler_service("Auth",      0.5),
        appeler_service("Stockage",  0.8),
        appeler_service("Recherche", 0.3),
    )

    total = time.time() - debut
    print(f"  Temps total parallele : {total:.2f}s")
    print(f"  Attendu : max(0.5, 0.8, 0.3) = 0.8s  -> 2x plus rapide !")
    return list(resultats)


async def main():
    res_seq  = await appels_sequentiels()
    res_para = await appels_paralleles()
    print(f"\nResultats seq  : {res_seq}")
    print(f"Resultats para : {res_para}")


if __name__ == "__main__":
    asyncio.run(main())
```

Listing 4 – async_demo.py — Appels séquentiels vs parallèles avec asyncio

5.5 Points pédagogiques

— `await asyncio.sleep(latence)` ne bloque pas le thread : il rend la main à la boucle d'événements, qui peut avancer d'autres coroutines en attente.

— Avec `gather`, les trois coroutines démarrent *simultanément* ; la première à terminer n'attend pas les autres.

— Le gain de $-50\,\%$ avec seulement 3 services augmente proportionnellement avec le nombre de services appelés.

A retenir

En distribué, **paralléliser les appels indépendants** est la première optimisation de latence à mettre en place. Pour plusieurs services de même latence, le gain correspond au nombre de services.

5.6 Résultat d'exécution

FIGURE 4 – Séquentiel : environ 1,6 s

6 Synthèse Comparative

Exemple	Pattern	Problème résolu	Quand l'utiliser
`serveur.py`	Client-serveur, JSON	Exposer une ressource via HTTP	Prototypage, enseignement
`client.py`	Timeout réseau	Éviter le blocage infini du thread	**Tout** appel réseau
`retry.py`	Retry + backoff + jitter	Tolérer les erreurs transitoires	Appels vers services critiques
`async_demo.py`	Parallélisme coopératif	Réduire la latence cumulée	Agrégation multi-services

6.1 Pièges fréquents à éviter

1. **Appel réseau sans timeout** → threads bloqués, ressources épuisées, cascade de pannes.
2. **Retry sans backoff** → DDoS involontaire sur le service fragilisé (*thundering herd*).
3. **Retry sur opération non idempotente** → doublons (virement, commande, enregistrement).
4. **Appels séquentiels pour des services indépendants** → latence additive évitable.
5. **Absence de gestion des codes 4xx/5xx** → comportement silencieusement incorrect.

7 Questions de Révision

Q1. Quelle est la différence entre un timeout et un retry ? Peut-on avoir l'un sans l'autre ?

Q2. Expliquer en une phrase pourquoi le jitter est indispensable dans une stratégie de backoff exponentiel.

Q3. Dans `async_demo.py`, que se passe-t-il si l'une des coroutines dans `gather` lève une exception ? Comment y remédier avec `return_exceptions=True` ?

Q4. Modifier `retry.py` pour qu'il **ne** réessaie **pas** sur un code HTTP 404, uniquement sur les codes 5xx et les timeouts.

Q5. Expliquer la notion d'idempotence et donner un exemple d'opération non idempotente dans le contexte du serveur de documents.

Références

— Tanenbaum, A.S. & Van Steen, M. (2017). *Distributed Systems : Principles and Paradigms* (3e éd.). Pearson.

— Documentation Python — `http.server` : https://docs.python.org/3/library/http.server.html
— Documentation Python — `asyncio` : https://docs.python.org/3/library/asyncio.html
— AWS Builder's Library — Timeouts, Retries, and Backoff : https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
— Deutsch, P. (1994). *The Eight Fallacies of Distributed Computing*. Sun Microsystems.